CareerAgent™ — Job Seeker Platform

Execution & Implementation Plan

HireOrGetHired (HOGH) | Version 1.0 | March 2026 | Confidential

---

Document Purpose

This document is the engineering and product execution plan for the **CareerAgent™** — the job seeker side of the HireOrGetHired platform. It is structured for use in development scoping, vendor selection, cost modelling, and sprint planning. Each module defines the feature in detail, the AI agents required (with their tasks, tools, and models), the integrations needed, the technical components to build, and the cost and complexity drivers.

Table of Contents

1. Platform Overview & Architecture Context
2. Module 1 — Career Intelligence Session (Onboarding)
3. Module 2 — Opportunity Discovery (DiscoveryAgent)
4. Module 3 — Application Engine (ApplicationAgent)
5. Module 4 — Opportunity Intelligence Brief™ (OpportunityIntelligenceAgent)
6. Module 5 — Interview Coaching (InterviewCoachAgent)
7. Module 6 — Offer Evaluation & Negotiation (NegotiationAdvisorAgent)
8. Module 7 — Career Path Modelling (CareerPathAgent)
9. Module 8 — NegotiationCoach™ Live Session (Candidate Side)
10. Cross-Cutting Infrastructure
11. Integration Registry — Full List
12. AI Agent Registry — Full List
13. Build Complexity & Cost Matrix
14. Development Roles Required
15. Phase-by-Phase Delivery Plan

---

1. Platform Overview & Architecture Context

What CareerAgent™ Is

CareerAgent™ is the job seeker's persistent AI "second brain." It operates continuously in the background — discovering opportunities, preparing tailored applications, briefing the user before every interview, and coaching through negotiation. The platform runs on both web (Next.js) and mobile (React Native) surfaces with a shared `@hogh/core` business logic layer.

Core Architectural Decisions (Relevant to Job Seeker Build)

Concern	Decision	Implication for Build
Frontend (Web)	Next.js 14 App Router	SSR for job discovery pages; requires Node.js server infra
Frontend (Mobile)	React Native + `@hogh/core`	Shared logic layer must be built before both apps
Agent Orchestration	LangGraph (Python + FastAPI)	All agents are LangGraph state machines — custom dev required
Primary DB	PostgreSQL (AWS RDS Aurora)	All application state, profile data, and audit logs
Vector DB	Pinecone	Semantic job matching, profile embeddings, brief source retrieval
Graph DB	Neo4j (Talent Intelligence Graph™)	Career path modelling, skill gap analysis, market signals
Cache	Redis (ElastiCache)	Session state, LLM response caching, rate limits
Auth	AWS Cognito	Consumer user auth; OAuth for LinkedIn/GitHub consent flows
Real-time	WebSocket (AWS API Gateway)	Live NegotiationCoach™ prompts, agent status streams

Concern	Decision	Implication for Build
Voice	WebRTC + AWS Transcribe Streaming	Interview coaching voice mode
Notifications	Firebase Cloud Messaging (FCM)	Interview reminders, offer alerts, agent completions
LLM Router	Internal `llm-gateway` service	Model selection, fallback, cost tracking, prompt versioning

---

2. Module 1 — Career Intelligence Session (Onboarding)

2.1 Feature Description

The Career Intelligence Session is the foundational onboarding experience. It is a 30–45 minute structured AI conversation that builds the user's complete profile: skills inventory, career motivations, work style preferences, compensation floor, location requirements, and short/long-term goals. Simultaneously, the system analyses the user's publicly available professional footprint (LinkedIn, GitHub, portfolio) to identify the gap between how the market currently perceives them and how they see themselves.

User-facing outputs:

- Completed career profile stored in CareerAgent™
- Skill gap matrix: visual map of current skills vs. target role requirements
- Learning pathway recommendations per skill gap
- "Public perception report" — how a recruiter would currently view their LinkedIn/GitHub

---

2.2 AI Agents Required

Agent 1.1 — `OnboardingConversationAgent`

Property	Detail
Role	Conducts the structured onboarding discovery conversation
Trigger	New user account creation; prompted to complete within 48 hours
LLM Model	GPT-4o (high conversation quality required; long context needed)
Orchestration	LangGraph state machine with defined conversation stages

Tasks:

1. Present structured discovery flow in natural language chat UI
2. Extract and structure: skills (with years of experience and proficiency level), past roles, companies, education, certifications
3. Elicit: career motivations, work style preferences (remote/hybrid/onsite), team size preference, industry preference
4. Establish: hard non-negotiables (comp floor, location constraints, must-have benefits)
5. Set: short-term goal (next role), long-term goal (5-year direction)
6. Collect: writing samples (optional) for voice calibration used later by ApplicationAgent
7. Validate and confirm structured profile before saving

Tools:

- `chat_interface_tool` — renders streaming conversational UI in Next.js app
- `profile_writer_tool` — writes structured extracted data to `candidate-service` PostgreSQL schema
- `skills_taxonomy_lookup_tool` — maps extracted skill strings to ESCO/O*NET canonical IDs in Neo4j
- `progress_tracker_tool` — tracks conversation stage completion; resumes incomplete sessions

Human Handoff: User reviews and confirms extracted profile before it is marked complete. No agent proceeds until user confirms.

Agent 1.2 — `PublicFootprintAnalysisAgent`

Property	Detail
Role	Analyses user's public professional presence and produces a "market perception report"

Property	Detail
Trigger	User grants OAuth consent for LinkedIn and/or GitHub; runs after OnboardingConversationAgent completes
LLM Model	GPT-4o (narrative synthesis)
Orchestration	Parallel async task dispatched by Master Orchestrator after onboarding completion

Tasks:

1. Fetch LinkedIn profile data via LinkedIn Talent Solutions API (OAuth 2.0 consent)
2. Fetch GitHub profile: public repos, contribution graph, README files, language breakdown via GitHub REST API
3. Fetch portfolio URL content if provided (web scrape via headless browser or Firecrawl API)
4. Embed each data source into Pinecone vector store under user's namespace
5. Generate "Public Perception Report":
    - How a recruiter's ATS/AI would currently score this profile
    - Top 3 skill signals that are strong
    - Top 3 perception gaps (skills user claims but has no public evidence for)
    - Profile completeness score against top 20% of profiles in same domain
6. Upsert candidate node and skill edges into Talent Intelligence Graph (Neo4j)

Tools:

- `linkedin_profile_fetch_tool` — calls LinkedIn Talent Solutions API; maps to internal schema
- `github_profile_fetch_tool` — calls GitHub REST API; parses repos, languages, contribution data
- `web_scraper_tool` — Firecrawl API for portfolio/personal website ingestion
- `pinecone_upsert_tool` — chunks, embeds (text-embedding-3-large), upserts to user namespace
- `neo4j_profile_upsert_tool` — creates/updates Candidate node, HAS_SKILL edges, WORKED_AT edges
- `perception_report_generator_tool` — calls GPT-4o with structured prompt; returns formatted report

---

Agent 1.3 — `SkillMapAgent`

Property	Detail
Role	Generates the skill gap matrix between current profile and target role
Trigger	After OnboardingConversationAgent and PublicFootprintAnalysisAgent both complete
LLM Model	Fine-tuned GPT-4o-mini (skill extraction and gap classification)
Orchestration	LangGraph node downstream of footprint analysis

Tasks:

1. Query Neo4j Talent Intelligence Graph: fetch `REQUIRES (Role → Skill)` edges for target role archetype with importance weights
2. Query user's `HAS_SKILL` edges from Neo4j
3. Compute gap: for each required skill, classify as `PRESENT`, `PARTIAL`, or `MISSING`
4. For each `MISSING` or `PARTIAL` skill, query learning pathway data (Coursera, LinkedIn Learning, Udemy course metadata)
5. Generate visual skill gap matrix (structured JSON → rendered as D3.js chart in frontend)
6. Produce learning pathway recommendations sorted by: (1) highest importance to target role, (2) fastest to acquire

Tools:

- `neo4j_skill_query_tool` — executes Cypher queries against Talent Intelligence Graph
- `learning_pathway_lookup_tool` — queries curated learning resource index (Coursera API, LinkedIn Learning API, internal curated data)
- `skill_gap_matrix_renderer_tool` — returns structured JSON for D3.js visual rendering in Next.js
- `gap_narrative_generator_tool` — calls GPT-4o-mini to produce readable gap summary with recommendations

---

2.3 Integrations Required

Integration	Purpose	Auth Method	API Type	Build Phase
LinkedIn Talent Solutions API	Fetch user's own profile data (with consent)	OAuth 2.0 (user grants consent)	REST	Phase 1
GitHub REST API	Fetch public repos, contribution history	OAuth 2.0 (user grants consent)	REST	Phase 1
Firecrawl API (or equivalent)	Scrape portfolio/personal websites	API Key	REST	Phase 1
OpenAI API (GPT-4o, text-embedding-3-large)	Conversation, analysis, embeddings	API Key	REST	Phase 1
Pinecone	Vector upsert and search	API Key	REST	Phase 1
Neo4j (Talent Intelligence Graph)	Skill taxonomy, career graph	Driver (Bolt protocol)	Graph query	Phase 1
Coursera API / LinkedIn Learning API	Learning pathway recommendations	API Key	REST	Phase 2

2.4 Technical Components to Build

Component	Type	Tech	Notes
Onboarding chat UI	Frontend	Next.js, SSE streaming	Multi-stage conversational flow with progress indicator
Onboarding resume parser	Backend service	Python + LlamaParse or PyMuPDF	Extract text from PDF/DOCX resume uploads
`OnboardingConversationAgent`	Agent	Python + LangGraph + FastAPI	Core LangGraph state machine
LinkedIn OAuth flow	Auth	AWS Cognito + LinkedIn OAuth 2.0	Consent screen, token storage in Secrets Manager
GitHub OAuth flow	Auth	AWS Cognito + GitHub OAuth 2.0	Same pattern as LinkedIn
Candidate profile schema	Database	PostgreSQL (Aurora)	Core user data model — all downstream agents depend on this
Skill gap matrix visualisation	Frontend	D3.js chart component	Requires custom SVG rendering
Learning pathway index	Database	PostgreSQL (curated) + external APIs	Curated initially; enriched via Coursera/LI Learning APIs
`PublicFootprintAnalysisAgent`	Agent	Python + LangGraph	Background async task
`SkillMapAgent`	Agent	Python + LangGraph	Downstream of footprint analysis
Profile edit UI	Frontend	Next.js	User can edit/correct extracted profile data

2.5 Build Complexity

Component	Complexity	Estimated Dev Effort
Chat UI + streaming	Medium	2 weeks (1 FE engineer)
OnboardingConversationAgent LangGraph state machine	High	3 weeks (1 AI/ML engineer)
LinkedIn + GitHub OAuth flows	Medium	1 week (1 backend engineer)
PublicFootprintAnalysisAgent	High	2 weeks (1 AI/ML engineer)
SkillMapAgent + D3 visualisation	High	3 weeks (1 AI/ML + 1 FE engineer)
Candidate profile schema + CRUD APIs	Low–Medium	1 week (1 backend engineer)

Component	Complexity	Estimated Dev Effort
Resume parser	Medium	1 week (1 backend engineer)

Phase: Module 1 is **Phase 1 foundational** — no other module can run without this. Prioritise for Sprint 1–3.

---

3. Module 2 — Opportunity Discovery (DiscoveryAgent)

3.1 Feature Description

DiscoveryAgent runs continuously in the background, scanning the live job market and surfacing roles matched to the user's full profile — not just job title keywords. Every surfaced role includes a transparent explanation of why it was matched. The user reviews the shortlist and explicitly approves which roles to pursue before any application begins.

User-facing outputs:

- Daily/real-time curated job shortlist with fit score and explanation
- Filter and search over shortlist (role type, salary, company size, location, remote)
- "Why matched" explanation panel for each role
- Exclusion controls: block specific companies, salary floors, role types
- Application pipeline entry point

3.2 AI Agents Required

Agent 2.1 — `DiscoveryAgent`

Property	Detail
Role	Continuous background job market scanner and match scorer
Trigger	Scheduled: runs every 4 hours per active user; also triggered on profile update
LLM Model	Fine-tuned GPT-4o-mini (JD parsing, skill extraction); GPT-4o (match explanation generation)
Orchestration	LangGraph scheduled task dispatched via AWS EventBridge → Kafka → `agent-orchestrator`

Tasks:

1. Query live job feeds from integrated sources (LinkedIn Jobs, Indeed, Greenhouse job boards, Lever job boards, direct company career pages via scraping)
2. For each new job posting: extract structured data (role title, skills required, seniority, compensation if listed, location, remote policy, company) using fine-tuned GPT-4o-mini
3. Map extracted skills to ESCO/O*NET taxonomy (using `skills_taxonomy_lookup_tool`)
4. Embed JD into Pinecone; run semantic similarity search against user's profile embedding
5. Query Neo4j: compute graph-based match score (GraphSAGE GNN model) incorporating career trajectory alignment
6. Apply user's exclusion rules: filter out blocked companies, sub-floor compensation, excluded role types
7. Apply deduplication: remove roles already seen by user in past 30 days
8. Generate fit score: weighted composite of (a) semantic similarity, (b) GNN graph match, (c) trajectory alignment
9. For top-N matches, call GPT-4o to generate a 2–3 sentence "why this role matches you" explanation
10. Deliver curated shortlist to user's discovery feed via WebSocket push + FCM push notification

Tools:

- `job_feed_aggregator_tool` — pulls from LinkedIn Jobs API, Indeed API, Greenhouse public API, Lever public API, Adzuna API
- `jd_parser_tool` — fine-tuned GPT-4o-mini to extract structured data from raw JD text
- `skills_taxonomy_lookup_tool` — ESCO/O*NET mapping via Neo4j lookup
- `pinecone_jd_embed_tool` — embeds JD chunks, upserts to shared JD namespace
- `pinecone_semantic_match_tool` — cosine similarity search between user embedding and JD embedding
- `neo4j_gnn_match_tool` — calls GraphSAGE model endpoint (SageMaker) for graph-based match score
- `exclusion_filter_tool` — applies user preference rules from `candidate-service`
- `dedup_tool` — Redis-backed seen-jobs cache per user (TTL 30 days)
- `match_explanation_generator_tool` — GPT-4o call to generate human-readable match rationale

- `discovery_feed_push_tool` — WebSocket delivery to dashboard + FCM push notification

---

3.3 Integrations Required

Integration	Purpose	Auth Method	API Type	Build Phase
LinkedIn Jobs API (Talent Solutions)	Live job postings	OAuth partner agreement	REST	Phase 1
Indeed Publisher API	Job postings	API Key (partner)	REST	Phase 1
Greenhouse Job Board API	Direct company job feeds	Public API	REST	Phase 1
Lever Job Postings API	Direct company job feeds	Public API	REST	Phase 2
Adzuna API	Broad market job aggregation	API Key	REST	Phase 2
Pinecone	Semantic job matching	API Key	REST	Phase 1
Neo4j	Graph-based match scoring (GNN)	Bolt driver	Graph	Phase 1
AWS SageMaker	GraphSAGE GNN inference endpoint	AWS SDK	REST	Phase 2
AWS EventBridge	Scheduled agent triggering	AWS SDK	Event	Phase 1
Firebase Cloud Messaging	Push notifications to mobile	Firebase Admin SDK	Push	Phase 1

3.4 Technical Components to Build

Component	Type	Tech	Notes
Job feed aggregator service	Backend microservice	Python + FastAPI	Multi-source ingestion with rate limit management
JD parser + skill extractor	AI model	Fine-tuned GPT-4o-mini	Requires fine-tuning dataset on ESCO taxonomy
Job postings database	Database	PostgreSQL + S3	Normalised job records; raw JD text in S3
Pinecone JD namespace	Vector DB	Pinecone	Separate namespace per indexed JD source
GNN match scoring pipeline	ML Pipeline	AWS SageMaker (GraphSAGE)	Phase 2 investment; Phase 1 uses vector similarity only
Discovery feed UI	Frontend	Next.js	Real-time feed, filter chips, fit score bars, "why matched" panel
User exclusion preferences UI	Frontend	Next.js	Company block list, salary floor, role type filters
Deduplication layer	Cache	Redis	Per-user seen-jobs set with 30-day TTL
FCM push notifications	Mobile	React Native + FCM	New matches, daily digest

3.5 Build Complexity

Component	Complexity	Estimated Dev Effort
Job feed aggregator (multi-source)	High	3 weeks (1 backend engineer)
JD parser fine-tuning pipeline	High	3 weeks (1 AI/ML engineer)
Semantic matching pipeline	Medium	2 weeks (1 AI/ML engineer)
GNN match scoring (Phase 2)	Very High	4 weeks (1 ML engineer)
Discovery feed UI	Medium	2 weeks (1 FE engineer)
Notifications (FCM)	Low	1 week (1 backend engineer)

---

4. Module 3 — Application Engine (ApplicationAgent)

4.1 Feature Description

For each role the user approves for application, ApplicationAgent generates a tailored resume (same career history, different emphasis per JD), a role-specific cover letter in the user's established writing voice, and autonomously completes multi-step application forms. Every output is reviewed and approved by the user before submission. Every application is tracked in a visual pipeline.

User-facing outputs:

- Tailored resume (PDF) per role — downloadable and editable
- Cover letter per role — editable before send
- Form-fill preview with user review step before submission
- Application tracker: status, timeline, follow-up prompts
- A/B performance data: which resume emphases get the most responses

---

4.2 AI Agents Required

Agent 3.1 — `ApplicationAgent`

Property	Detail
Role	Generates tailored application materials and automates form submission
Trigger	User approves a role in the discovery feed
LLM Model	Claude 3.5 Sonnet (resume tailoring — strong at instruction-following while preserving user voice); GPT-4o-mini (form-fill extraction)
Orchestration	LangGraph task triggered by user approval event via Kafka

Tasks:

Resume Tailoring:

1. Retrieve user's master profile and career history from `candidate-service`
2. Retrieve and parse the target JD (from job postings DB)
3. Identify top-N required skills from JD (via skill extractor)
4. Reorder and reframe user's bullet points to emphasise most relevant experiences for this specific JD — using Claude 3.5 Sonnet for voice preservation
5. Inject relevant keywords from JD naturally into experience descriptions (ATS optimisation)
6. Generate formatted resume in structured JSON → render to PDF via headless Puppeteer/WeasyPrint
7. Return resume to user for review and optional editing
8. On user approval, store as final version linked to this application

Cover Letter Generation:

1. Pull company intelligence from Opportunity Intelligence Brief™ if pre-generated (or trigger brief pre-generation)
2. Generate cover letter: opening that references a specific, verifiable company signal; body connecting user's top 2–3 experiences to JD requirements; closing with clear call to action
3. Write in user's established voice (learned from onboarding writing samples + past letters)
4. Return to user for review and edit

Form-Fill Automation:

1. Detect application form type: direct company careers page, Greenhouse form, Lever form, Workday apply, iCIMS
2. Use Puppeteer (headless browser) to navigate the application form
3. Map profile fields to form fields via GPT-4o-mini extraction
4. Populate all fields; flag any fields where data is missing from profile
5. Present pre-submission review screen to user showing all populated fields
6. On user confirmation: submit form; capture submission confirmation

Tools:

- `profile_retriever_tool` — fetches full candidate profile from `candidate-service`
- `jd_retriever_tool` — fetches parsed JD from job postings DB
- `resume_tailor_tool` — Claude 3.5 Sonnet call with resume tailoring prompt template
- `resume_pdf_renderer_tool` — Puppeteer/WeasyPrint to render structured JSON to PDF
- `cover_letter_generator_tool` — Claude 3.5 Sonnet call with cover letter prompt template
- `form_detector_tool` — classifies application form type (Greenhouse/Lever/Workday/custom)
- `form_filler_tool` — Puppeteer headless browser automation for form population
- `form_field_mapper_tool` — GPT-4o-mini to map profile data to form field structure
- `application_tracker_writer_tool` — writes new application record to PostgreSQL application tracker

---

Agent 3.2 — `ApplicationTrackerAgent`

Property	Detail
Role	Monitors application statuses and surfaces follow-up recommendations
Trigger	Scheduled daily; also triggered by email webhook events (SendGrid inbound)
LLM Model	GPT-4o-mini (email parsing, status classification)
Orchestration	Scheduled EventBridge job + email inbound webhook

Tasks:

1. Parse inbound email responses from companies (interview invites, rejections, requests for more info)
2. Update application status in tracker
3. If no response after configurable window (default: 10 business days), suggest follow-up action
4. Surface analytics: response rate by role type, resume version performance, company sector response rates
5. Alert user via FCM push on status changes

Tools:

- `email_parser_tool` — GPT-4o-mini to extract status signal from email text
- `application_status_updater_tool` — updates application record in PostgreSQL
- `follow_up_recommender_tool` — generates follow-up email draft for user review
- `application_analytics_tool` — aggregates response rate data for dashboard

4.3 Integrations Required

Integration	Purpose	Auth Method	Build Phase
Anthropic API (Claude 3.5 Sonnet)	Resume tailoring, cover letter generation	API Key	Phase 1
OpenAI API (GPT-4o-mini)	Form field mapping, email parsing	API Key	Phase 1
Puppeteer / Playwright (headless browser)	Form-fill automation on company career pages	Self-hosted	Phase 1
WeasyPrint / Puppeteer	PDF generation for tailored resumes	Self-hosted	Phase 1
Greenhouse Apply API	Direct form submission to Greenhouse ATS	REST API	Phase 1
Lever Apply API	Direct form submission to Lever	REST API	Phase 2
SendGrid Inbound Parse	Parse inbound email responses from companies	Webhook	Phase 1
AWS SES	Outbound transactional emails	AWS SDK	Phase 1

4.4 Technical Components to Build

Component	Type	Tech	Notes
Resume builder/editor UI	Frontend	Next.js + draft-js or TipTap	In-browser editable resume with tracked changes from agent
Resume PDF renderer	Backend	Puppeteer (Node.js)	Server-side rendering to PDF; must handle layout fidelity

Component	Type	Tech	Notes
Cover letter editor UI	Frontend	Next.js + TipTap	Rich text editing with agent suggestions highlighted
Form-fill automation engine	Backend	Python + Playwright	Containerised headless browser; proxy routing per submission
Application tracker UI	Frontend	Next.js	Kanban or table view; status badges; follow-up prompts
Application database schema	Database	PostgreSQL	One row per application; links to resume version, cover letter, JD
Email inbound parse webhook	Backend	Node.js + SendGrid Inbound Parse	Classify and update application status
Voice calibration store	Database	PostgreSQL + S3	User writing samples for voice preservation

4.5 Build Complexity

Component	Complexity	Estimated Dev Effort
Resume tailoring agent (Claude)	High	2 weeks (1 AI/ML engineer)
Resume editor + PDF renderer	High	3 weeks (1 FE + 1 backend engineer)
Form-fill automation (Playwright)	Very High	4 weeks (1 backend engineer) — brittle; requires ongoing maintenance
Cover letter generator	Medium	1 week (1 AI/ML engineer)
Application tracker UI	Medium	2 weeks (1 FE engineer)
Email inbound parse	Medium	1 week (1 backend engineer)

Risk note: Form-fill automation across arbitrary company career pages is the highest-risk component — websites change structure frequently. Mitigation: prioritise native integrations (Greenhouse Apply API, Lever API) first; use headless browser only as fallback.

---

5. Module 4 — Opportunity Intelligence Brief™ (OpportunityIntelligenceAgent)

5.1 Feature Description

Delivered 24 hours before every interview. A comprehensive, role-specific briefing document covering: company culture intelligence, interviewer profile, role market context, narrative optimisation talking points, anticipated interview questions with suggested responses, and honest gap analysis with framing strategies.

User-facing outputs:

- Full Opportunity Intelligence Brief™ document (readable in-app, offline-capable via React Query cache)
- Interviewer profile cards with communication style notes
- Salary range widget for the specific role in specific geography
- "Your 5 talking points" section tailored to user's background
- "Likely questions" section with suggested answers
- "Honest gaps" section with how-to-address guidance

5.2 AI Agents Required

Agent 4.1 — `OpportunityIntelligenceAgent`

Property	Detail
Role	Master orchestrator for Opportunity Intelligence Brief generation
Trigger	Interview scheduled (SchedulerAgent event); or user manually triggers "prepare for interview"
LLM Model	GPT-4o (deep synthesis; multi-source narrative generation)
Orchestration	LangGraph pipeline dispatching 5 parallel sub-tasks

Tasks (Master Orchestrator Level):

1. Receive trigger: (user_id, company_id, role_id, interview_datetime, interviewer_names[])
2. Decompose into 5 parallel sub-agent tasks (see below)
3. Await all sub-agent completions (with timeout + partial-brief fallback if one source fails)
4. Synthesise all sub-agent outputs into unified Opportunity Intelligence Brief™ using GPT-4o
5. Structure output as typed JSON (Pydantic schema); validate output
6. Store brief in PostgreSQL (linked to application record); push to offline cache
7. Send FCM push notification to user: "Your interview brief is ready"

Sub-tasks dispatched to specialist agents:

Agent 4.1a — `CompanyIntelligenceSubAgent`

Tasks:

1. Query Glassdoor Partner API for company reviews, interview reviews, rating data
2. Query Crunchbase API: funding stage, headcount, recent news, investor names
3. Query Bing Web Search API: recent news articles about the company (last 90 days), product launches, layoffs, leadership changes
4. Fetch company's LinkedIn page data (employee growth trend, recent posts) via LinkedIn API
5. RAG retrieval: retrieve most relevant chunks from Pinecone (previously indexed company data)
6. Synthesise into: culture narrative, growth signals, red flags, mission/values alignment with user's stated preferences

Tools:

- `glassdoor_company_fetch_tool` — Glassdoor Partner API
- `crunchbase_company_fetch_tool` — Crunchbase Basic API
- `bing_news_search_tool` — Bing Web Search API (filtered to company domain + name)
- `linkedin_company_fetch_tool` — LinkedIn Company API
- `pinecone_retrieval_tool` — semantic retrieval from company data namespace
- `company_intel_synthesiser_tool` — GPT-4o synthesis call

Agent 4.1b — `InterviewerProfileSubAgent`

Tasks:

1. Fetch interviewer LinkedIn profiles via LinkedIn API (public data, with user consent)
2. Search for interviewer public content: talks (YouTube API search), published articles (Medium, Substack, Google Scholar)
3. Infer communication style and priorities from writing (topic analysis, tone classification)
4. Map interviewer's professional background to role context
5. Generate: 2–3 specific interviewer-relevant talking points for the user; things this interviewer is known to care about

Tools:

- `linkedin_profile_fetch_tool` — interviewer public profile data
- `youtube_search_tool` — YouTube Data API v3 search for interviewer name + topic
- `web_search_tool` — Bing search for articles, papers by interviewer name
- `content_style_analyser_tool` — GPT-4o-mini tone and topic analysis
- `interviewer_profile_synthesiser_tool` — GPT-4o synthesis call

Agent 4.1c — `RoleMarketContextSubAgent`

Tasks:

1. Query Talent Intelligence Graph: salary benchmark for this exact role + geography + seniority (p25/median/p75/p90)
2. Query Levels.fyi data (if tech role): comp bands with equity and bonus breakdown
3. Query Bureau of Labor Statistics API: occupational employment and wage statistics for role category
4. Compute: demand trend for required skills (skill scarcity index from Neo4j MarketSignal nodes)
5. Generate: role market context narrative (is this role in high demand? is comp competitive?)

Tools:

- `neo4j_salary_benchmark_tool` — queries `BENCHMARK (Role → MarketSignal)` edges in Neo4j
- `levelsfyi_data_tool` — licensed Levels.fyi data API
- `bls_api_tool` — Bureau of Labor Statistics public API
- `market_context_synthesiser_tool` — GPT-4o-mini narrative generation

Agent 4.1d — `NarrativeOptimisationSubAgent`

Tasks:

1. Cross-reference user's profile (skills, experiences, story) with JD requirements
2. Identify 3–5 specific experiences from user's history that best match what the role needs
3. Generate "bridge statements" — how to connect each experience to the role's stated needs
4. Produce 5 curated talking points specific to: (a) this user, (b) this company, (c) this role

Tools:

- `profile_retriever_tool` — user profile from `candidate-service`
- `jd_retriever_tool` — parsed JD
- `narrative_optimiser_tool` — GPT-4o prompt: "given this profile and this JD, generate 5 specific talking points"

Agent 4.1e — `QuestionPredictionSubAgent`

Tasks:

1. Retrieve question banks from: Glassdoor interview Q&A for this company, role-specific common questions (curated database), interviewer's known focus areas (from Agent 4.1b)
2. Generate role-specific anticipated questions using GPT-4o (based on JD + company + interviewer)
3. For each question: generate a suggested answer grounded in the user's actual experience (not generic advice)
4. Generate "gap questions" — questions the user is likely to struggle with, with honest framing strategies

Tools:

- `glassdoor_interview_qa_tool` — Glassdoor Partner API interview reviews
- `question_bank_retrieval_tool` — Pinecone semantic search over curated question library
- `question_generator_tool` — GPT-4o: generates novel predicted questions from JD + company + interviewer context
- `answer_generator_tool` — GPT-4o: generates user-specific suggested answers grounded in their profile
- `gap_framer_tool` — GPT-4o: identifies weak areas and generates honest framing strategies

---

5.3 Integrations Required

Integration	Purpose	Auth Method	Build Phase
Glassdoor Partner API	Company reviews, interview Q&A, salary data	Partner agreement	Phase 1
Crunchbase Basic API	Company funding, news, headcount	API Key	Phase 1
Bing Web Search API	Recent company news, interviewer content	API Key (Azure Cognitive Services)	Phase 1
LinkedIn API (Company + Profile)	Company data, interviewer profiles	OAuth partner	Phase 1
YouTube Data API v3	Interviewer public talks and content	API Key	Phase 2
Levels.fyi	Tech comp benchmarks	Data licensing agreement	Phase 2
Bureau of Labor Statistics API	Wage statistics	Public API	Phase 1
Neo4j	Salary benchmarks from Talent Intelligence Graph	Bolt driver	Phase 1
Pinecone	Semantic retrieval of company data, Q&A	API Key	Phase 1

5.4 Build Complexity

Component	Complexity	Estimated Dev Effort
OpportunityIntelligenceAgent LangGraph pipeline	Very High	4 weeks (1 AI/ML engineer)
CompanyIntelligenceSubAgent	High	2 weeks (1 AI/ML engineer)
InterviewerProfileSubAgent	High	2 weeks (1 AI/ML engineer)
RoleMarketContextSubAgent	Medium	1 week (1 AI/ML engineer)
NarrativeOptimisationSubAgent	Medium	1 week (1 AI/ML engineer)
QuestionPredictionSubAgent	High	2 weeks (1 AI/ML engineer)
Brief UI (offline-capable)	High	2 weeks (1 FE engineer)
Brief delivery + offline cache	Medium	1 week (1 FE + 1 backend engineer)

---

6. Module 5 — Interview Coaching (InterviewCoachAgent)

6.1 Feature Description

Personalised mock interview sessions using questions generated from the actual JD, the interviewer's known focus areas, and the user's profile. Real-time feedback on content, conciseness, and STAR structure. Voice mode with spoken mock interview, transcription, and annotated feedback. Session history tracks improvement over time.

User-facing outputs:

- Mock interview session (text or voice mode)
- Real-time scoring: content quality, STAR adherence, conciseness, filler word count
- Session transcript with annotation
- Improvement trends dashboard: which question types are improving, which need more work
- "Starred" answers saved for pre-interview review

6.2 AI Agents Required

Agent 5.1 — `InterviewCoachAgent`

Property	Detail
Role	Conducts personalised mock interview sessions; provides real-time feedback
Trigger	User launches coaching session from dashboard (linked to a specific application)
LLM Model	GPT-4o (question generation, answer evaluation, feedback synthesis)
Orchestration	Real-time LangGraph session with WebSocket/WebRTC loop

Tasks:

Question Generation:

1. Pull questions from OpportunityIntelligenceAgent brief (if generated) — use pre-computed predicted questions
2. If no brief available: generate fresh questions from JD + company context
3. Mix: role-specific technical/competency questions, behavioural questions, "gap" questions the user historically struggles with (from session history)
4. Dynamically adapt follow-up questions based on user's answer to previous question

Answer Evaluation (Text Mode):

1. Receive user's typed response
2. Score on 5 dimensions: (a) relevance to question, (b) STAR structure adherence, (c) specificity (are claims concrete?), (d) conciseness (word count efficiency), (e) keyword alignment with JD
3. Generate annotated feedback: highlight strong parts, flag vague claims, suggest specific improvements

Answer Evaluation (Voice Mode):

1. Capture WebRTC audio stream from user's browser/device
2. Stream to AWS Transcribe Streaming for real-time transcription
3. Analyse transcription: detect filler words (um, uh, like, you know), pacing (words per minute), pauses
4. After user finishes: evaluate transcribed answer on same 5 dimensions as text mode
5. Return: audio annotation markers + written feedback panel

Session Management:

1. Track all questions asked and answers given in this session
2. After session: generate session summary — strengths, areas to improve, specific questions to practice more
3. Update session history in PostgreSQL
4. Identify question type patterns the user struggles with → feed back to next session's question mix

Tools:

- `question_generator_tool` — GPT-4o call: generates session questions from JD + brief + session history
- `answer_scorer_tool` — GPT-4o evaluation call with structured scoring rubric (Pydantic output schema)
- `webrtc_audio_capture_tool` — WebRTC audio stream from browser (React Native handles mobile)
- `aws_transcribe_streaming_tool` — streams audio to AWS Transcribe; returns real-time transcript
- `filler_word_detector_tool` — regex + ML classifier on transcript text
- `feedback_generator_tool` — GPT-4o call: synthesises evaluation into human-readable coaching feedback
- `session_history_writer_tool` — writes session record to PostgreSQL
- `weak_area_tracker_tool` — updates user's question-type performance model in PostgreSQL

6.3 Integrations Required

Integration	Purpose	Auth Method	Build Phase
AWS Transcribe Streaming	Real-time voice transcription	AWS SDK	Phase 2
WebRTC	Browser/mobile audio capture	Browser API + React Native	Phase 2
OpenAI API (GPT-4o)	Question generation, answer evaluation, feedback	API Key	Phase 1
AWS EventBridge	Session scheduling and reminders	AWS SDK	Phase 1

6.4 Technical Components to Build

Component	Type	Tech	Notes
Interview session UI (text mode)	Frontend	Next.js	Chat-like interface; question cards; answer input; live scoring
Interview session UI (voice mode)	Frontend	Next.js + WebRTC	Microphone capture; real-time transcription display; waveform visualiser
Real-time feedback panel	Frontend	Next.js	Side panel with live scoring bars
Session history dashboard	Frontend	Next.js	Line charts for improvement trends; session replay
Voice transcription pipeline	Backend	Python + AWS Transcribe Streaming	WebSocket relay from frontend to Transcribe
Session record schema	Database	PostgreSQL	Questions, answers, scores, feedback per session per user

---

6.5 Build Complexity

Component	Complexity	Estimated Dev Effort
Text mode interview UI + agent	High	3 weeks (1 FE + 1 AI/ML engineer)
Voice mode (WebRTC + Transcribe)	Very High	4 weeks (1 FE + 1 backend engineer)

Component	Complexity	Estimated Dev Effort
Answer scoring + feedback generation	High	2 weeks (1 AI/ML engineer)
Session history + improvement tracking	Medium	2 weeks (1 FE + 1 backend engineer)

Note: Voice mode is a Phase 2 feature. Text mode launches first in Phase 1.

---

7. Module 6 — Offer Evaluation & Negotiation (NegotiationAdvisorAgent)

7.1 Feature Description

When a user receives an offer, NegotiationAdvisorAgent benchmarks it against live market data, models the user's BATNA (best alternative to a negotiated agreement), generates a specific recommended counter-offer with framing language, and identifies which components are worth pushing on vs. accepting. Activates the live NegotiationCoach™ session when the user is about to have a compensation conversation.

User-facing outputs:

- Offer analysis card: offer vs. market p25/median/p75/p90
- BATNA model: what your leverage actually is
- Recommended counter-offer: specific number with confidence range and reasoning
- Negotiation script: exact language suggestions for specific scenarios
- "What's negotiable here" guide: equity vs. base vs. benefits vs. signing bonus based on company pattern data
- NegotiationCoach™ activation button

7.2 AI Agents Required

Agent 6.1 — `NegotiationAdvisorAgent`

Property	Detail
Role	Analyses offer, models BATNA, generates counter-offer recommendation and negotiation briefing
Trigger	User logs an offer received (manual entry or parsed from email)
LLM Model	GPT-4o (negotiation strategy reasoning; game-theoretic scenario planning)
Orchestration	On-demand LangGraph task triggered by user action

Tasks:

1. Parse offer details: base salary, bonus, equity (type, cliff, vesting), benefits, signing bonus, start date, remote policy
2. Query Neo4j Talent Intelligence Graph: salary benchmark for role + location + seniority (p25/median/p75/p90)
3. Query Levels.fyi (tech roles): total comp breakdown benchmarks
4. Query platform aggregate data: company's known offer flexibility range (anonymised; from platform historical data)
5. Model BATNA: does user have other active offers? What is their walk-away floor? What is current employed status?
6. Compute negotiation leverage score: composite of (market position, BATNA strength, company's historical flexibility)
7. Generate recommended counter-offer:
    - Specific figure (not a range) with probabilistic confidence interval
    - Which component to counter on first (base, equity, signing, remote)
    - Framing language: exact sentence-level script for opening the negotiation conversation
8. Generate "What's negotiable" breakdown: each offer component classified as (Likely negotiable / Sometimes negotiable / Rarely negotiable) based on company-specific data
9. Generate scenario responses: if company says "this is our final offer" → specific response; if they ask "what are you looking for" → specific response

Tools:

- `offer_parser_tool` — extracts structured offer data from email or user input form
- `neo4j_salary_benchmark_tool` — salary percentile query from Talent Intelligence Graph
- `levelsfyi_comp_tool` — Levels.fyi licensed data query
- `platform_offer_history_tool` — anonymised aggregate query on company's historical offer distribution

- `batna_modeller_tool` — GPT-4o structured reasoning on user's leverage position
- `counter_offer_generator_tool` — GPT-4o: generates specific counter-offer recommendation
- `negotiation_script_generator_tool` — GPT-4o: generates scenario-specific negotiation language

---

7.3 Integrations Required

Integration	Purpose	Build Phase
Neo4j	Salary benchmarks	Phase 1
Levels.fyi	Tech comp benchmarks	Phase 2
Glassdoor Partner API	Salary data	Phase 1
SendGrid Inbound Parse	Parse offer emails	Phase 2

7.4 Build Complexity

Component	Complexity	Estimated Dev Effort
NegotiationAdvisorAgent (LangGraph)	High	2 weeks (1 AI/ML engineer)
Offer analysis UI	Medium	2 weeks (1 FE engineer)
BATNA modeller	High	1 week (1 AI/ML engineer)
Negotiation script UI	Medium	1 week (1 FE engineer)

---

8. Module 7 — Career Path Modelling (CareerPathAgent)

8.1 Feature Description

Multi-year career trajectory simulation. Given the user's current profile and stated goals, models 3–4 plausible career paths with projected compensation, title progression, and required skill development. Role ROI comparison: if choosing between two offers, models 5-year financial and career-capital implications. Market trend overlay highlights trajectories being accelerated or disrupted by technology trends.

User-facing outputs:

- Career path visualisation: 3–4 branching paths with projected titles and compensation bands per year
- Required skill development per path: what to learn to unlock each trajectory
- Role ROI comparison: side-by-side 5-year financial and career-capital model for two offers
- Market trend overlay: which paths are accelerating vs. disrupting based on current skill demand data

---

8.2 AI Agents Required

Agent 7.1 — `CareerPathAgent`

Property	Detail
Role	Simulates multi-year career trajectories and models offer ROI
Trigger	User opens Career Path section; or triggered when user has 2+ active offers to compare
LLM Model	GPT-4o (career narrative reasoning); fine-tuned GPT-4o-mini (trajectory pattern classification)
Orchestration	On-demand LangGraph task

Tasks:

1. Retrieve user's current profile: title, skills, years of experience, industry, location, stated goals
2. Query Neo4j: `PRECEDED_BY (Role → Role)` edges — common career transitions from current role type
3. Query Neo4j: `BENCHMARK (Role → MarketSignal)` — compensation bands for each potential next role
4. Query Neo4j `MarketSignal` nodes: demand trend (accelerating/stable/declining) for skills required by each path

5. Model 3–4 distinct career paths:
    - Path 1: Linear continuation (same domain, upward)
    - Path 2: Adjacent pivot (related domain, lateral then upward)
    - Path 3: Strategic stretch (target goal role, requires skill gap bridging)
    - Path 4: Market-driven (highest demand trajectory given current skills)
6. For each path: model year-by-year title, compensation, required skill development
7. If user has 2+ active offers: generate role ROI comparison model (5-year NPV of total comp + career capital value)
8. Generate narrative: explain trade-offs between paths in plain language

Tools:

- `neo4j_career_path_query_tool` — Cypher query on `PRECEDED_BY` and `BENCHMARK` edges
- `neo4j_market_signal_tool` — skill demand trend data from `MarketSignal` nodes
- `gnn_path_probability_tool` — GraphSAGE model: probability score for each career transition
- `career_path_modeller_tool` — GPT-4o structured reasoning call; outputs Pydantic path model
- `role_roi_calculator_tool` — 5-year NPV computation (Python function, not LLM)
- `career_path_visualiser_tool` — returns structured JSON for D3.js branching tree visualisation

---

8.3 Build Complexity

Component	Complexity	Estimated Dev Effort
CareerPathAgent (LangGraph)	High	3 weeks (1 AI/ML engineer)
Career path visualisation (D3.js)	High	2 weeks (1 FE engineer)
Role ROI calculator	Medium	1 week (1 backend engineer)
Market trend overlay	Medium	1 week (1 AI/ML engineer)

---

9. Module 8 — NegotiationCoach™ Live Session (Candidate Side)

9.1 Feature Description

During the live offer conversation between the candidate and recruiter, NegotiationCoach™ provides real-time private coaching prompts to the candidate through their mobile app or browser. Both parties must acknowledge AI advisory is active before the session begins. The candidate sees: live offer benchmarking, recommended counter-offer, "which benefits are negotiable here," and dynamically updating coaching prompts as the conversation develops.

Activation requirement: Both candidate and recruiter must acknowledge NegotiationCoach™ is active. Mutual acknowledgment is verified and logged before session activates.

9.2 AI Agents Required

Agent 8.1 — `NegotiationCoachCandidateAgent`

Property	Detail
Role	Delivers real-time private negotiation coaching to candidate during live offer conversation
Trigger	Both parties acknowledge NegotiationCoach™ activation; session link shared
LLM Model	GPT-4o (real-time if-then scenario reasoning; must deliver advice in < 800ms p95)
Orchestration	Persistent WebSocket session; event-driven update loop

Tasks:

1. Pre-session: load offer data, market benchmarks, BATNA model (from NegotiationAdvisorAgent pre-computation)
2. Establish private WebSocket channel for this candidate (separate from recruiter channel — no data shared between channels)
3. Display: offer vs. market benchmark, recommended counter, negotiable components
4. Accept real-time input from candidate: "they said X" — candidate types or speaks the recruiter's latest move
5. Update coaching prompts in real-time based on conversation state:

    - If recruiter pushes back on base → recommend pivoting to equity cliff extension
    - If recruiter says "final offer" → recommended response with reasoning
    - If recruiter asks for a number first → recommended anchor strategy
6. Log all session events to audit log (immutable)
7. Session end: generate negotiation summary with outcome and learning points

Tools:

- `websocket_candidate_stream_tool` — private WebSocket channel (candidate side only)
- `offer_benchmark_display_tool` — real-time benchmark data display
- `scenario_advisor_tool` — GPT-4o: given conversation state, generate next best action
- `audit_log_writer_tool` — writes every session event to immutable audit log
- `session_summariser_tool` — GPT-4o: generates post-session summary

Latency SLO: Advisory response to candidate input must be < 800ms p95. Uses GPT-4o with streaming response; prompt is pre-loaded with context to minimise input token count.

9.3 Technical Components to Build

Component	Type	Tech	Notes
NegotiationCoach™ session UI (candidate)	Frontend	Next.js + React Native	Real-time prompt feed; offer benchmark panel; input field
Private WebSocket channel management	Backend	Python + FastAPI + AWS API Gateway WebSocket	Strict channel isolation — no cross-contamination between recruiter and candidate channels
Session acknowledgment flow	Frontend + Backend	Next.js + PostgreSQL	Mutual acknowledgment gate; logged to audit
Real-time scenario advisor	Agent	Python + LangGraph + GPT-4o	Sub-800ms requirement; pre-loaded context
Audit log for session	Database	PostgreSQL (append-only)	Every event timestamped and immutable

---

10. Cross-Cutting Infrastructure

10.1 `candidate-service` — Core Backend Service

The central service managing all job seeker data. All agents read from and write to this service.

Responsibilities:

- Candidate profile CRUD (skills, experience, preferences, goals)
- Application tracking records
- Session history (interview coaching)
- CareerAgent™ state machine persistence
- DSAR processing (data export + deletion requests)
- Row-level security: each user's data is isolated

Tech: Node.js + Express + PostgreSQL (Aurora RDS, Multi-AZ)

10.2 `agent-orchestrator` — Master Orchestrator Service

Responsibilities:

- Receives tasks from `candidate-service` and other services via Kafka
- Instantiates and manages LangGraph state machines for each user task
- Enforces Human Handoff Protocol checkpoints
- Streams agent progress to frontend via SSE
- Handles retries, failures, and escalation

Tech: Python + FastAPI + LangGraph + Kafka

10.3 `llm-gateway` — LLM Routing Service

Responsibilities:

- Routes LLM calls to correct model (GPT-4o, Claude 3.5 Sonnet, GPT-4o-mini)
- Fallback routing if primary model unavailable
- Prompt versioning (stored in `prompt-registry`)
- Response caching (Redis — 7-day TTL for identical prompt+context pairs)
- Cost tracking per user and per tenant
- A/B testing of prompt variants

Tech: Python + FastAPI + Redis + PostgreSQL (prompt registry)

10.4 Authentication & User Management

Component	Tech	Notes
Consumer auth	AWS Cognito	Email/password + Google SSO for consumer users
LinkedIn OAuth	AWS Cognito + LinkedIn OAuth 2.0	Token stored in Secrets Manager; scoped to profile read only
GitHub OAuth	AWS Cognito + GitHub OAuth 2.0	Token stored in Secrets Manager; scoped to public data only
MFA	AWS Cognito TOTP	Optional for consumer; recommended in onboarding

---

10.5 Offline Capability

Opportunity Intelligence Briefs are downloadable for offline reading. Implementation:

- React Query persistent cache (localStorage on web; SQLite on mobile via React Native MMKV)
- Briefs serialised as structured JSON → rendered client-side
- FCM push triggers cache pre-warm 24 hours before interview

---

11. Integration Registry — Full List

#	Integration	Module(s)	Auth	API Type	Cost Model	Build Phase
1	LinkedIn Talent Solutions API	1, 2, 4	OAuth 2.0 (partner)	REST	Per API call (partner pricing)	Phase 1
2	GitHub REST API	1	OAuth 2.0 (user consent)	REST	Free (rate limited)	Phase 1
3	OpenAI API (GPT-4o, GPT-4o-mini, text-embedding-3-large)	All	API Key	REST	Per token (usage-based)	Phase 1
4	Anthropic API (Claude 3.5 Sonnet)	3	API Key	REST	Per token (usage-based)	Phase 1
5	Pinecone	1, 2, 4	API Key	REST	Per vector/query (usage-based)	Phase 1
6	Neo4j (Talent Intelligence Graph)	1, 2, 4, 6, 7	Bolt driver	Graph	Self-hosted or AuraDB cloud	Phase 1
7	AWS SageMaker	2, 7	AWS SDK	REST	Per inference hour	Phase 2
8	AWS Transcribe Streaming	5	AWS SDK	WebSocket	Per audio second	Phase 2
9	AWS Cognito	All	AWS SDK	Managed	Per MAU	Phase 1

#	Integration	Module(s)	Auth	API Type	Cost Model	Build Phase
10	AWS SES	3, 6	AWS SDK	SMTP/REST	Per email	Phase 1
11	Firebase Cloud Messaging	2, 4, 5, 6	Firebase Admin SDK	Push	Free (usage-based)	Phase 1
12	Indeed Publisher API	2	API Key (partner)	REST	Per job post/click	Phase 1
13	Greenhouse Job Board API	2, 3	Public REST	REST	Free	Phase 1
14	Lever Job Postings API	2, 3	Public REST	REST	Free	Phase 2
15	Adzuna API	2	API Key	REST	Per request	Phase 2
16	Glassdoor Partner API	4, 6	Partner agreement	REST	Partner pricing	Phase 1
17	Crunchbase Basic API	4	API Key	REST	Per request	Phase 1
18	Bing Web Search API	4	Azure API Key	REST	Per transaction	Phase 1
19	Levels.fyi	4, 6	Data license	REST	Licensed (annual fee)	Phase 2
20	Bureau of Labor Statistics API	4, 6	Public	REST	Free	Phase 1
21	YouTube Data API v3	4	API Key	REST	Per quota unit	Phase 2
22	SendGrid (Outbound + Inbound Parse)	3	API Key	REST/Webhook	Per email	Phase 1
23	Puppeteer/Playwright	3	Self-hosted	Library	Infrastructure cost	Phase 1
24	WeasyPrint/Puppeteer	3	Self-hosted	Library	Infrastructure cost	Phase 1
25	Coursera API	1	API Key	REST	Partner pricing	Phase 2
26	LinkedIn Learning API	1	OAuth (partner)	REST	Partner pricing	Phase 2
27	Firecrawl API	1	API Key	REST	Per URL crawled	Phase 1
28	AWS EventBridge	2, 5	AWS SDK	Event	Per event	Phase 1
29	Apache Kafka (MSK)	All	AWS MSK	Event stream	Per broker hour	Phase 1
30	Redis (ElastiCache)	All	AWS SDK	Cache	Per node hour	Phase 1

12. AI Agent Registry — Full List

#	Agent Name	Module	LLM Model	Primary Tasks	Key Tools	Orchestration
1	`OnboardingConversationAgent`	1	GPT-4o	Structured discovery conversation; profile extraction	chat_interface, profile_writer, skills_taxonomy_lookup	LangGraph state machine
2	`PublicFootprintAnalysisAgent`	1	GPT-4o	LinkedIn/GitHub analysis; perception report generation	linkedin_fetch, github_fetch, pinecone_upsert, neo4j_upsert	LangGraph async task
3	`SkillMapAgent`	1	GPT-4o-mini	Skill gap matrix; learning pathway recommendations	neo4j_skill_query, learning_pathway_lookup, gap_matrix_renderer	LangGraph downstream node
4	`DiscoveryAgent`	2	GPT-4o-mini + GPT-4o	Job market scanning; match scoring; shortlist generation	job_feed_aggregator, jd_parser, pinecone_match, neo4j_gnn_match, dedup	LangGraph scheduled task
5	`ApplicationAgent`	3	Claude 3.5 Sonnet + GPT-4o-mini	Resume tailoring; cover letter; form-fill automation	resume_tailor, pdf_renderer, form_filler, cover_letter_generator	LangGraph triggered by user approval
6	`ApplicationTrackerAgent`	3	GPT-4o-mini	Email status parsing; follow-up recommendations	email_parser, application_status_updater, follow_up_recommender	Scheduled + webhook
7	`OpportunityIntelligenceAgent`	4	GPT-4o	Master brief orchestration; 5-sub-agent synthesis	brief_synthesiser, delivery_tool	LangGraph pipeline
8	`CompanyIntelligenceSubAgent`	4	GPT-4o	Company culture synthesis	glassdoor_fetch, crunchbase_fetch, bing_search, pinecone_retrieval	Sub-agent of Module 4
9	`InterviewerProfileSubAgent`	4	GPT-4o	Interviewer background; communication style analysis	linkedin_profile_fetch, youtube_search, web_search, style_analyser	Sub-agent of Module 4
10	`RoleMarketContextSubAgent`	4	GPT-4o-mini	Salary benchmark; demand trend	neo4j_salary_benchmark, levelsfyi_data, bls_api	Sub-agent of Module 4
11	`NarrativeOptimisationSubAgent`	4	GPT-4o	Talking points generation; experience bridging	profile_retriever, jd_retriever, narrative_optimiser	Sub-agent of Module 4
12	`QuestionPredictionSubAgent`	4	GPT-4o	Question prediction; user-specific answer generation	glassdoor_qa, question_bank_retrieval, question_generator, answer_generator	Sub-agent of Module 4

#	Agent Name	Module	LLM Model	Primary Tasks	Key Tools	Orchestration
13	`InterviewCoachAgent`	5	GPT-4o	Mock interview sessions; real-time feedback; voice mode	question_generator, answer_scorer, aws_transcribe, filler_word_detector, feedback_generator	LangGraph real-time session
14	`NegotiationAdvisorAgent`	6	GPT-4o	Offer analysis; BATNA modelling; counter-offer generation	offer_parser, salary_benchmark, platform_offer_history, batna_modeller, counter_offer_generator	On-demand LangGraph task
15	`CareerPathAgent`	7	GPT-4o + GPT-4o-mini	Career trajectory simulation; offer ROI comparison	neo4j_career_path_query, gnn_path_probability, career_path_modeller, roi_calculator	On-demand LangGraph task
16	`NegotiationCoachCandidateAgent`	8	GPT-4o	Live real-time negotiation coaching	websocket_stream, scenario_advisor, audit_log_writer	Persistent WebSocket session

13. Build Complexity & Cost Matrix

13.1 Development Complexity by Module

Module	Frontend Complexity	Backend Complexity	AI/Agent Complexity	Overall	Recommended Phase
1 — Onboarding	Medium	Medium	High	**High**	Phase 1 (Month 1–3)
2 — Discovery	Medium	High	High	**High**	Phase 1 (Month 2–4)
3 — Application Engine	High	Very High	High	**Very High**	Phase 1 (Month 3–5)
4 — OI Brief™	High	High	Very High	**Very High**	Phase 1 (Month 3–5)
5 — Interview Coaching	High	High	High	**High**	Phase 1 text / Phase 2 voice
6 — Offer Evaluation	Medium	Medium	High	**High**	Phase 2 (Month 5–7)
7 — Career Path	High	Medium	High	**High**	Phase 2 (Month 6–8)
8 — NegotiationCoach™ Live	High	Very High	High	**Very High**	Phase 2 (Month 7–9)

13.2 Recurring Operational Cost Drivers (Monthly, at Scale)

Cost Category	Driver	Estimated Cost Range (at 10k MAU)	Notes
LLM API — GPT-4o	OI Brief synthesis, coaching feedback, negotiation advisor	$8,000–$15,000/month	Brief = ~50k tokens; coaching session = ~20k tokens
LLM API — GPT-4o-mini	JD parsing, form-fill, email parsing, skill extraction	$1,000–$3,000/month	High volume; low cost per call
LLM API — Claude 3.5 Sonnet	Resume tailoring, cover letters	$2,000–$5,000/month	Per application ~5k tokens

Cost Category	Driver	Estimated Cost Range (at 10k MAU)	Notes
Pinecone	Vector storage and query	$500–$2,000/month	Scales with indexed JD volume
Neo4j AuraDB	Talent Intelligence Graph hosting	$2,000–$5,000/month	Professional tier; scales with node count
AWS Transcribe Streaming	Voice coaching mode	$500–$2,000/month	Per audio minute
AWS infrastructure (EKS, RDS, ElastiCache, SageMaker)	All services	$3,000–$8,000/month	Scales with compute usage
External data APIs (Glassdoor, Crunchbase, Bing, LinkedIn)	OI Brief generation	$2,000–$6,000/month	Glassdoor and LinkedIn are largest cost items
Firebase Cloud Messaging	Push notifications	Negligible	Free tier sufficient at 10k MAU
SendGrid	Email delivery	$200–$500/month	Per email volume

Total estimated operational cost at 10k MAU: ~$19,000–$46,000/month (Significant optimisation available via LLM caching, batching, and fine-tuning — see product_note.md Section 7.4)

13.3 One-Time Build Cost Drivers

Item	Estimated Engineering Cost	Notes
Talent Intelligence Graph initial data build (Neo4j + ESCO/O*NET import)	$30,000–$50,000	One-time data engineering effort; ongoing enrichment cheaper
GraphSAGE GNN model training	$20,000–$40,000	Requires curated training data; Phase 2 investment
Fine-tuning GPT-4o-mini for JD parsing	$10,000–$20,000	Training data curation + fine-tuning runs
LinkedIn Talent Solutions partnership setup	$0–$10,000 (legal + BD time)	LinkedIn partnership agreement required
Glassdoor data partnership	$0–$5,000 (legal + BD time)	Glassdoor partner agreement required
Levels.fyi data license	$10,000–$30,000/year	Annual license fee

---

14. Development Roles Required

Role	Modules	FTE Estimate (Phase 1)	Notes
AI/ML Engineer	All agent modules	2–3 FTE	LangGraph, LLM integration, RAG pipeline, fine-tuning
Backend Engineer (Python)	Agent orchestrator, intelligence services, data pipelines	2 FTE	FastAPI, Kafka, LangGraph, PostgreSQL
Backend Engineer (Node.js)	candidate-service, notification-service, integrations	1–2 FTE	Express, PostgreSQL, external API integrations
Frontend Engineer (Next.js)	All web UI	2 FTE	Next.js 14 App Router, React Query, D3.js, TipTap
React Native Engineer	Mobile app	1 FTE	Shared `@hogh/core` with web
ML Engineer (Graph/GNN)	Talent Intelligence Graph, GraphSAGE	1 FTE (Phase 2)	Neo4j, GraphSAGE, SageMaker

Role	Modules	FTE Estimate (Phase 1)	Notes
Data Engineer	Data pipeline, Pinecone, S3/Athena, Kafka	1 FTE	ETL pipelines, embedding pipeline
DevOps / Platform Engineer	AWS EKS, CI/CD, observability	1 FTE	Kubernetes, Helm, Terraform, monitoring stack
Security Engineer	Auth, compliance, encryption	0.5 FTE (contractor)	AWS Cognito, KMS, GDPR/EU AI Act
Product Designer	All UI surfaces	1 FTE	Design system, UX flows, mobile/web

Minimum Phase 1 team: 10–12 engineers + 1 designer

---

15. Phase-by-Phase Delivery Plan

Phase 1 — Core Platform (Months 1–6)

Goal: Deliver a working CareerAgent™ that onboards users, discovers jobs, generates applications, and prepares interview briefs. All core agents operational in text mode.

Sprint	Deliverable	Modules
Sprint 1–2	Infrastructure: EKS cluster, RDS, Redis, Kafka, Cognito auth, CI/CD pipeline	Cross-cutting
Sprint 3–4	Candidate profile schema, onboarding UI, OnboardingConversationAgent	Module 1
Sprint 5–6	LinkedIn + GitHub OAuth, PublicFootprintAnalysisAgent, SkillMapAgent	Module 1
Sprint 7–8	Job feed aggregator, DiscoveryAgent (v1 — semantic matching only), discovery feed UI	Module 2
Sprint 9–10	ApplicationAgent (resume tailoring + cover letter), resume editor UI	Module 3
Sprint 11–12	Form-fill automation (Greenhouse + Lever native first), application tracker	Module 3
Sprint 13–14	OpportunityIntelligenceAgent pipeline (all 5 sub-agents), brief delivery UI	Module 4
Sprint 15–16	InterviewCoachAgent (text mode), session history	Module 5
Sprint 17–18	Integration testing, performance optimisation, bug fix sprint, soft launch	All

---

Phase 2 — Intelligence Depth & Mobile (Months 7–12)

Goal: Deepen intelligence quality, add voice mode, launch mobile app, add offer negotiation and career path features.

Sprint	Deliverable	Modules
Sprint 19–20	GNN match scoring (SageMaker GraphSAGE), DiscoveryAgent upgrade	Module 2
Sprint 21–22	Voice mode InterviewCoachAgent (WebRTC + Transcribe), React Native mobile app	Module 5
Sprint 23–24	NegotiationAdvisorAgent, offer evaluation UI	Module 6
Sprint 25–26	CareerPathAgent, career path visualisation	Module 7
Sprint 27–28	NegotiationCoach™ live session (candidate side)	Module 8
Sprint 29–30	Levels.fyi data integration, YouTube interviewer research, learning platform integrations	Modules 4, 5

Phase 3 — Scale & Compliance (Months 13–18)

Goal: EU expansion, EU AI Act compliance, enterprise-grade security, API tier, and fine-tuned models.

Sprint	Deliverable
Sprint 31–34	GDPR data residency (eu-west-1 deployment), DSAR processing automation

Sprint	Deliverable
Sprint 35–36	EU AI Act transparency tooling (candidate data disclosure, opt-out mechanism)
Sprint 37–38	GPT-4o-mini fine-tuning for skill extraction (reduce GPT-4o dependency)
Sprint 39–40	Performance hardening, SLO monitoring, load testing at scale

---

End of CareerAgent™ Execution & Implementation Plan Document version 1.0 | March 2026 | HireOrGetHired | Confidential